BACK-END · NODE.JS · TYPESCRIPT · FASTIFY · POSTGRESQL

Qirsh قرش

Expense & Budget Tracking REST API

Every fils accounted for.

Product & Design Specification · Version 1.0 · October 2026

Prepared for Obieda Al-Darrabsih · Software engineering portfolio project

Scope: portfolio / demonstration build, not a production service

00 Contents

Qirsh is a REST API for tracking expenses and budgets, built with Node.js, TypeScript, Fastify and PostgreSQL. It has no user interface of its own on purpose: it is a back-end portfolio piece whose "product" is a clean, documented, tested and rate-limited API that any web or mobile client could use.

01 Overview
02 API Consumers & Roles
03 Core Workflows
04 Feature Scope
05 Brand Identity & Logo
06 Design System (Docs & DX)
07 API Style Guide
08 Docs Portal Mockup
09 Architecture & Stack
10 Data Model
11 Endpoint Reference
12 The Hard Problems
13 Security
14 Testing Strategy
15 Observability
16 DevOps & Repository
17 Build Plan
18 README, Limitations & CV
19 Interview Preparation

01 Overview

Product	**Qirsh · قرش** — expense & budget REST API
Category	Back-end (Node.js + TypeScript), OpenAPI-first
Tagline	Every fils accounted for.
Build target	`docker compose up` → API, PostgreSQL, Redis, interactive docs

Users create a **workspace** (a household, a small team, a freelancer's business), record expenses and income in categories, set monthly budgets, and pull reports: monthly totals, spending by category, budget versus actual, and trends. Expenses can be imported in bulk from CSV, scheduled as recurring, and exported. Access is by user tokens or by scoped API keys for scripts.

Why this project

A back-end developer is judged by what the API guarantees: correct money arithmetic, predictable errors, safe retries, fair rate limits, honest documentation and tests that prove it. Qirsh is intentionally narrow in domain so that each of those guarantees is visible and testable.

Proves #1 · Money done right
Integer minor units, per-currency exponents (JOD = 3), deterministic rounding, month boundaries in the workspace's time zone.

Proves #2 · API craft
OpenAPI generated from the same schemas that validate requests, RFC 9457 errors, cursor pagination, idempotency keys, versioning.

Proves #3 · Operability
Redis-backed rate limits, structured logs with request IDs, metrics, health checks, integration tests on real Postgres.

SCOPE HONESTY

No bank connections, no real financial data, no currency conversion. Seed data is fictional. The README must list these limits.

02 API Consumers & Roles

Consumer / role	Typical use	Permissions
Owner	Creates the workspace and invites others.	Everything, incl. delete workspace, manage members and API keys.
Member	Family member or teammate logging expenses.	Create/edit own expenses, read all expenses and reports, manage budgets if allowed by owner.
Viewer	Accountant or partner who reviews.	Read-only: expenses, reports, exports.
API key	A script, spreadsheet or bot that adds expenses.	Scoped: `expenses:write`, `reports:read`, etc. Never manages members or keys.
Client app (future)	A web/mobile front-end built later.	Uses user tokens; the API is designed so no extra endpoints are needed.

03 Core Workflows

A. Record an expense

1. `POST /v1/workspaces/:id/transactions` with amount (minor units), currency, category, date, note.
2. Schema validation (Zod) → 400 with field-level problems if invalid.
3. Business rules: currency must equal the workspace currency; category must belong to the workspace; date not more than 1 day in the future.
4. Insert + audit row in one transaction; respond 201 with the resource and `Location` header.

B. Budget and report

1. Owner sets `PUT /budgets/2026-10/categories/:categoryId` with a limit.
2. Client asks for `GET /reports/budget-vs-actual?month=2026-10`.
3. Server computes the month range in the workspace time zone (Asia/Amman by default), sums by category with one SQL query, and returns spent, remaining and percent used.

C. Bulk import from CSV

1. Client uploads CSV to `POST /imports` with an `Idempotency-Key`.
2. Server streams and validates each row; returns 202 with an import id.
3. Worker inserts valid rows in batches; invalid rows are reported with line numbers.
4. `GET /imports/:id` shows progress, inserted count and row errors.
5. Re-sending the same request with the same key returns the same import, never duplicates.

D. Recurring expenses

A rule (e.g. rent, monthly on day 1) is stored with a simple schedule. A daily job creates due transactions with a deterministic key `(rule_id, occurrence_date)`, so a job that runs twice cannot create the rent twice.

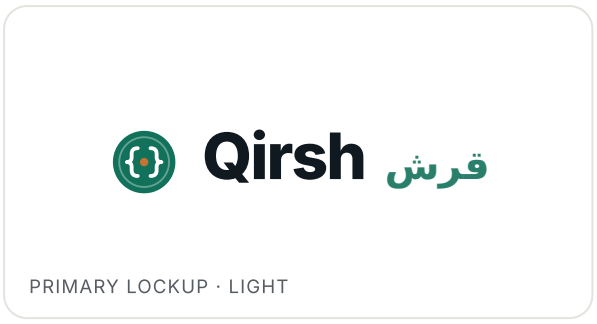
04 Feature Scope

Feature	Description	Release
Auth	Register, login, access + refresh tokens with rotation, logout.	MVP
Workspaces & members	Create, invite (token link), roles owner/member/viewer.	MVP
Categories	Default seeded set (Groceries, Rent, Transport ...), custom categories, archive.	MVP
Transactions	CRUD, filters (date range, category, amount range, text), cursor pagination, soft delete.	MVP
Budgets	Monthly limit per category; copy last month's budgets.	MVP
Reports	Monthly summary, by category, budget vs actual, 12-month trend.	MVP
OpenAPI docs	Generated spec + interactive docs at `/docs`.	MVP
Rate limiting	Per user / per API key / per IP, Redis-backed.	MVP

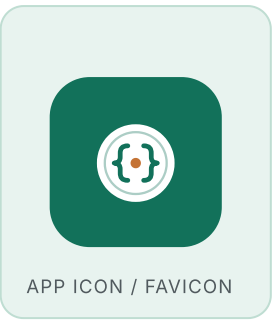
API keys	Scoped keys, shown once, hashed at rest, last-used tracking.	Stretch
CSV import / export	Async import with row errors; streamed export.	Stretch
Recurring rules	Monthly / weekly rules with an idempotent job.	Stretch
Bank sync, OCR receipts, multi-currency conversion	Large features with external dependencies.	Out of scope

05 Brand Identity & Logo

Qirsh (قرش) is the everyday Levantine word for a small coin, as in *every qirsh counts*. It suits an API whose whole job is to make sure no amount is ever lost to rounding. The mark is a coin whose face is a pair of curly braces, the universal sign of JSON and code, with a copper dot at the centre: money, as data. Because this is a back-end product, the identity lives in the API documentation portal, the README, error pages and terminal output rather than an app.







Construction

64 × 64 grid. Coin radius 27; inner rim radius 21.5 at 35% opacity. Braces drawn with a 3.3 stroke, round joins, 11 units apart. Copper dot radius 3.6, exact centre.

Where it appears

Docs portal header, Swagger UI favicon, README banner (1280 × 320), social preview image, terminal banner on server start (ASCII-safe text version: `{ qirsh }`).

Don'ts

Do not add a currency symbol inside the coin (the API is currency-agnostic), do not use copper for small text on white, and do not animate the braces.

Repo files: `docs/brand/qirsh-mark.svg`, `qirsh-lockup.svg`, `qirsh-lockup-dark.svg`, `favicon.svg`, `readme-banner.png`, `social-preview.png` (1280 × 640).

Color Palette

Contrast ratios are calculated against white (#FFFFFF) and the brand ink (#101820). WCAG AA needs 4.5:1 for body text and 3:1 for large text and UI parts.

Ledger Green
#12715A · Primary
Logo, links, primary buttons in the docs portal, '2xx' badges.
on white 5.9:1 · on ink 3.0:1

Vault Black
#0E1A17 · Dark
Docs sidebar, code panels, dark logo tile.
on white 17.8:1 · on ink 1.0:1

Copper
#C47335 · Accent
Coin dot in logo, 'deprecated' tag, chart highlight. Large text only on white.
on white 3.6:1 · on ink 5.0:1

Paper
#F7F7F3 · Background
Docs portal background.
on white 1.1:1 · on ink 16.7:1

Pure White
#FFFFFF · Surface
Cards, request/response panels.
on white 1.0:1 · on ink 17.9:1

Ink
#101820 · Text
Body text.
on white 17.9:1 · on ink 1.0:1

Slate
#56606A · Muted text
Parameter descriptions, captions.
on white 6.4:1 · on ink 2.8:1

GET Blue
#2459B8 · Method: GET
HTTP method badge.
on white 6.6:1 · on ink 2.7:1

POST Green
#1D7D46 · Method: POST
HTTP method badge, success.
on white 5.2:1 · on ink 3.5:1

PATCH Amber
#9A6400 · Method: PATCH
HTTP method badge, warnings.
on white 5.0:1 · on ink 3.6:1

DELETE Red
#C0362C · Method: DELETE
HTTP method badge, errors, 4xx/5xx.
on white 5.5:1 · on ink 3.2:1

Mint Wash
#E8F3EF · Tint
Callouts, selected endpoint.
on white 1.1:1 · on ink 15.8:1

06 Design System (Docs & Developer Experience)

For an API, design means: **how the docs look, how errors read, and how the terminal feels.** Mood: a clean ledger book. Paper background, ink text, green for "money in order", copper for emphasis. Code panels are dark so examples stand out.

Typography

Docs heading	**Budget vs actual**	Inter 800 · 28/34
Body	Compares each category's budget with spending.	Inter 400 · 15/24
Endpoint	`GET /v1/reports/budget-vs-actual`	JetBrains Mono 500 · 14/20
Code sample	`"spentMinor": 152750`	JetBrains Mono 400 · 13/20
Method badge	GET POST PATCH DELETE	Mono 700 · 11 · caps

Docs portal components

Endpoint page

Left: summary, auth scope, parameters table, response codes. Right (sticky): request example (curl / fetch tabs) and response example. Built with Scalar or Redoc themed via CSS variables, served at `/docs`.

Guides

Quick start, Money & currencies, Errors, Pagination, Idempotency, Rate limits. Each guide ends with a runnable example against the local Docker stack.

Error page format

Every error code has a docs page (`/docs/errors/budget-exceeded`) and the Problem Details `type` URL points to it.

Terminal & logs

On start: one-line banner `{ qirsh } v1 · http://localhost:3000 · docs /docs`. Logs are JSON in production, pretty-printed in dev (pino-pretty) with green 2xx, amber 4xx, red 5xx.

07 API Style Guide

Topic	Convention
URLs	Plural nouns, kebab-case: `/v1/workspaces/:workspaceId/transactions`. Version in the path (`/v1`). Max two levels of nesting.
JSON	camelCase keys; UTF-8; dates ISO 8601 with offset (`2026-10-18T09:30:00+03:00`); calendar dates as `YYYY-MM-DD`; months as `YYYY-MM`.
Money	`amountMinor` (integer) + `currency` (ISO 4217). Never floats, never strings with symbols. Optional `amountFormatted` for display only.
Pagination	Cursor-based: `?limit=50&cursor=…` → `{ "data": [...], "nextCursor": "…" | null }`. Cursor encodes `(occurred_on, id)`; stable under inserts.

Filtering & sorting	`from`, `to`, `categoryId`, `minAmount`, `maxAmount`, `q`; `sort=-occurredOn` (prefix `-` for descending).
Errors	RFC 9457 Problem Details, content type `application/problem+json`, with an extra `code` and, for validation, an `errors[]` list of `{ path, message }`.
Status codes	200 read/update · 201 create (+ `Location`) · 202 accepted (async) · 204 delete · 400 validation · 401 · 403 · 404 · 409 conflict · 412 stale `If-Match` · 422 business rule · 429 rate limited.
Concurrency	Resources return an `ETag`; `PATCH` accepts `If-Match` to prevent lost updates.
Idempotency	`Idempotency-Key` header accepted on all `POST`s, required on `/imports`; stored 24 h.

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{ "type": "https://qirsh.dev/docs/errors/currency-mismatch",
  "title": "Currency does not match workspace",
  "status": 422, "code": "CURRENCY_MISMATCH",
  "detail": "Workspace uses JOD; got USD.", "instance": "req_01J9…" }
```

08 Docs Portal Mockup

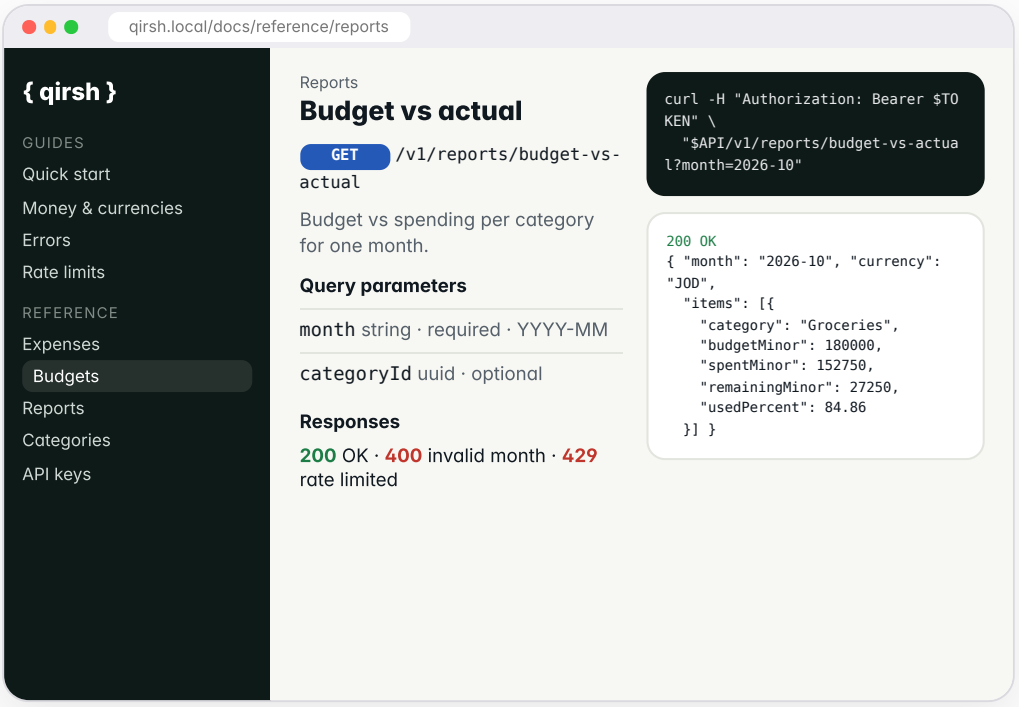
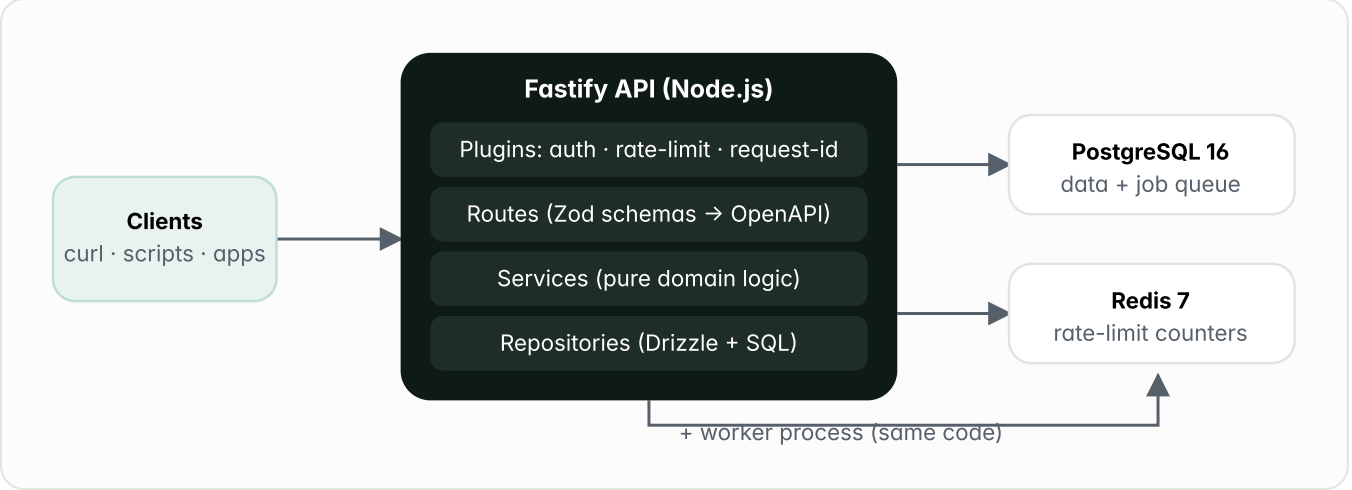


Fig. 1 — Reference page for a report endpoint: dark navigation, parameters and responses on the left, copy-ready request and response on the right.

Docs page	Content
Home	What Qirsh is, 3-step quick start, links to guides and reference.
Quick start	Run with Docker, register, create workspace, add expense, get report — all with curl.
Money & currencies	Minor units table (JOD 3, USD 2, JPY 0), rounding rule, examples.
Errors	Problem Details format and the full list of `code` values with fixes.
Rate limits	Limits per plan, headers, how to back off on 429.
Reference	Generated from the OpenAPI spec; one page per tag.
Changelog	Versioned list of API changes; deprecations marked with the copper tag.

09 Architecture & Tech Stack



Layer	Choice	Why
Runtime	Node.js (current LTS) + TypeScript strict	Typed end to end; ESM.
HTTP	Fastify	Fast, schema-first, plugin system with clean encapsulation; `inject()` for tests without a port.
Schemas	Zod + `fastify-type-provider-zod`	One schema gives validation, TypeScript types and the OpenAPI spec.
Docs	`@fastify/swagger` + Scalar / Redoc	Spec at `/openapi.json`, themed portal at `/docs`.
Database	PostgreSQL 16 + Drizzle ORM + drizzle-kit migrations	SQL-like, typed queries; easy to drop to raw SQL for reports.
Jobs	pg-boss (queue in Postgres)	No extra infrastructure; retries and scheduling built in.
Rate limiting	`@fastify/rate-limit` with Redis store	Shared counters across instances.
Auth	`@fastify/jwt`, Argon2id (`@node-rs/argon2`)	Short-lived access tokens; rotating refresh tokens stored hashed.
Logging / metrics	pino, `prom-client`	Structured JSON logs with request IDs; Prometheus metrics at `/metrics`.
Tests	Vitest, Testcontainers, k6	See §14.

10 Data Model

Table	Key columns	Constraints & notes
users	id, email, password_hash, name, created_at	email unique (citext).
refresh_tokens	id, user_id, token_hash, family_id, expires_at, revoked_at	Reuse of a revoked token revokes the whole family.
workspaces	id, name, currency, timezone, created_by	currency CHAR(3); timezone IANA, default `Asia/Amman`.
memberships	workspace_id, user_id, role	PK (workspace_id, user_id); role enum.
categories	id, workspace_id, name, kind (EXPENSE/INCOME), color, archived_at	unique (workspace_id, lower(name)).
transactions	id, workspace_id, category_id, kind, amount_minor BIGINT, currency, occurred_on DATE, note, created_by, version, deleted_at	CHECK amount_minor > 0; index (workspace_id, occurred_on DESC, id); `version` for ETag.
budgets	workspace_id, category_id, month DATE (first day), limit_minor	PK (workspace_id, category_id, month).
recurring_rules	id, workspace_id, template (jsonb), frequency, day, next_run_on, active	Generated rows carry `(rule_id, occurrence_on)` unique.
imports	id, workspace_id, status, total_rows, inserted, failed, errors jsonb	Status: QUEUED → RUNNING → DONE / FAILED.
api_keys	id, workspace_id, name, prefix, key_hash, scopes text[], last_used_at, revoked_at	Shown once; SHA-256 hash stored; prefix for lookup.
idempotency_keys	principal_id, key, request_hash, status, response jsonb, created_at	PK (principal_id, key); purged after 24 h.
audit_log	id, workspace_id, actor, action, entity, entity_id, diff jsonb, at	Append-only.

Why DATE for `occurred_on`? An expense belongs to a calendar day in the workspace's time zone, not to an instant. Storing a timestamp would move late-night expenses into the wrong day (or month) when converted to UTC.

11 Endpoint Reference

All paths are prefixed with `/v1`. Workspace routes are under `/workspaces/:wid` (shortened as `…` below) and require membership.

Method	Path	Purpose	Scope / role
POST	`/auth/register`, `/auth/login`	Create account / get tokens	public
POST	`/auth/refresh`, `/auth/logout`	Rotate / revoke refresh token	user
GET/POST	`/workspaces`	List mine / create	user
POST	`…/invites`, `/invites/:token/accept`	Invite member / accept	owner / user

GET/POST	`…/categories`	List / create category	member
GET/POST	`…/transactions`	Filtered list (cursor) / create	viewer / member
GET/PATCH/DELETE	`…/transactions/:id`	Read / update (If-Match) / soft delete	member (own)
GET/PUT	`…/budgets/:month`	All budgets for a month / set many	viewer / owner
POST	`…/budgets/:month/copy-from/:prev`	Copy last month's limits	owner
GET	`…/reports/summary?month=`	Income, expenses, net	viewer
GET	`…/reports/by-category?from=&to=`	Totals and share per category	viewer
GET	`…/reports/budget-vs-actual?month=`	Budget, spent, remaining, % used	viewer
GET	`…/reports/trend?months=12`	Monthly totals series	viewer
POST/GET	`…/imports`, `…/imports/:id`	Start CSV import / status	member
GET	`…/exports/transactions.csv`	Streamed export	viewer
GET/POST/DELETE	`…/api-keys`	Manage scoped keys	owner
GET	`/health/live`, `/health/ready`, `/metrics`	Liveness, readiness (DB + Redis), Prometheus	public / internal

12 The Hard Problems

12.1 Money without rounding bugs

Floating point cannot represent 0.1 exactly, so sums drift. Qirsh stores `amount_minor` as a 64-bit integer and keeps a small table of currency exponents (JOD 3, USD 2, JPY 0). Parsing "15.25" in JOD yields `15250`; "15.2505" is rejected (too many decimals) rather than silently rounded. When a value must be split (for example a yearly budget into 12 months), the remainder is distributed one minor unit at a time to the first months, so the parts always add up to the total. Percentages in reports are computed at the end and rounded once to two decimals.

12.2 Which month is "October"?

For a workspace in Asia/Amman (UTC+3), an expense entered at 01:30 on 1 November local time is still 31 October in UTC. Because `occurred_on` is a `DATE` chosen in the workspace's zone, month reports are simple range queries (`occurred_on >= '2026-10-01' AND occurred_on < '2026-11-01'`) with no conversion, and a test covers the boundary.

12.3 Fair rate limiting across instances

Counters live in Redis, so two API instances share one limit. Keys are chosen by principal: API key id, else user id, else client IP. Defaults: 300 requests/min per user, 60/min per API key, 20/min per IP on `/auth/*`. Responses carry `x-ratelimit-limit`, `x-ratelimit-remaining`, `x-ratelimit-reset`, and a 429 includes `Retry-After`. If Redis is down, the limiter fails open for reads and closed for `/auth` (documented trade-off).

12.4 Safe retries and duplicate-free jobs

```
// Idempotency, simplified
const saved = await idem.begin(principal, key, hash(req.body));   // INSERT … ON CONFLICT DO NOTHING
if (saved.replay)   return reply.code(saved.status).send(saved.body);
if (saved.mismatch) throw problem(422, 'IDEMPOTENCY_KEY_REUSED');
const result = await imports.start(...);
await idem.finish(principal, key, 202, result);
```

Recurring generation relies on a unique constraint instead of careful timing: inserting `(rule_id, occurrence_on)` twice fails harmlessly, so the job can be retried or run on two workers.

12.5 Lost updates

Two members editing the same transaction: each `GET` returns `ETag: "v7"`. A `PATCH` with `If-Match: "v7"` updates `WHERE id = $1 AND version = 7`; if someone else saved first, 0 rows change and the API answers `412 Precondition Failed`.

13 Security

Area	Measure
Authentication	Argon2id; access token 15 min; refresh token 30 days, rotated on every use, stored hashed; reuse detection revokes the family.
Authorization	Membership + role checked in a Fastify `preHandler`; every repository query filters by `workspace_id`; tests try to read other workspaces' IDs and expect 404.

API keys	Random 32 bytes, shown once, SHA-256 stored, scopes enforced per route, revocable, `last_used_at` tracked.
Input	Zod on body, query, params and headers; body size limit 1 MB (10 MB for imports); CSV parsed as data, formulas in exported cells escaped (CSV injection).
Transport & headers	`@fastify/helmet`, strict CORS allow-list, HTTPS assumed behind a proxy (`trustProxy` configured).
Secrets & logs	`.env` gitignored; `.env.example` committed. pino redacts `authorization`, `password`, `token` fields.

14 Testing Strategy

Level	What	Tools
Unit	Money parsing and splitting (property-based: parts always sum to the total), month ranges, CSV row validation, ETag logic.	Vitest + fast-check
Integration	Each route through `app.inject()` against real PostgreSQL and Redis in containers; migrations applied fresh per run.	Vitest + Testcontainers
Authorization matrix	Table-driven test: every route × every role (owner, member, viewer, API key with/without scope, stranger) → expected status.	Vitest
Contract	Generated OpenAPI is valid; responses in tests are validated against it; the spec is snapshot-tested so breaking changes show up in review.	openapi validator
Concurrency	Two PATCHes with the same ETag → one 200, one 412. Same import key twice in parallel → one import.	Vitest
Load / limits	k6 script proves 429s start at the configured limit and recover after reset.	k6

> **HEADLINE TEST**
>
> `authz.matrix.test.ts` — about 100 generated cases in one readable table. It is the clearest proof that tenants are isolated, and it is easy to show in an interview.

15 Observability

- **Request IDs:** accepted from `x-request-id` or generated; returned in responses and in Problem Details `instance`.
- **Logs:** one JSON line per request (method, route, status, duration, principal type), never bodies of auth routes.
- **Metrics:** request duration histogram by route and status, rate-limit rejections, job queue depth, DB pool usage.
- **Health:** `/health/live` (process up) and `/health/ready` (DB and Redis reachable) for container orchestration.

16 DevOps & Repository

```
qirsh/
├ src/
│  ├ app.ts            # buildApp() for server + tests
│  ├ plugins/          # auth, rate-limit, errors, docs
│  ├ modules/
│  │  ├ transactions/  # routes, schemas, service, repo
│  │  ├ budgets/  reports/  imports/
│  │  └ auth/  workspaces/  api-keys/
│  ├ lib/money.ts  months.ts  problem.ts
│  └ db/schema.ts  migrations/  seed.ts
├ worker/              # pg-boss jobs
├ test/ unit/ integration/ load/
├ docs/ guides/*.md  brand/  decisions/
├ docker-compose.yml   # api, worker, db, redis
├ Dockerfile · .env.example · LICENSE
└ .github/workflows/ci.yml
```

Commands

- `docker compose up --build` → API :3000, docs :3000/docs, seeded demo workspace.
- `npm test`, `npm run test:load`, `npm run db:migrate`.

CI

- Typecheck, lint, unit, integration (service containers), OpenAPI validation, spec diff comment on PRs.
- Build and push Docker image tagged with the commit SHA (GitHub Container Registry).

Docs hosting

Export the OpenAPI spec and guides to a static site on GitHub Pages so reviewers can read the docs without running anything.

17 Build Plan (≈ 3–4 weeks, part-time)

PHASE 1 · DAYS 1–3

Skeleton

- Fastify + TS, Zod provider, error plugin (Problem Details)
- Drizzle schema, migrations, Docker Compose, CI

Done when: `/health/ready` is green in Docker and CI.

PHASE 2 · DAYS 4–6

Auth & workspaces

- Register/login/refresh rotation, memberships, roles

Done when: refresh-token reuse revokes the family (test).

PHASE 3 · DAYS 7–10

Transactions

- Money lib, CRUD, filters, cursor pagination, ETags

Done when: property tests on money pass; 412 test passes.

PHASE 4 · DAYS 11–13

Budgets & reports

- Monthly budgets, four reports, time-zone boundary tests

Done when: 31 Oct 23:30 Amman counts in October.

PHASE 5 · DAYS 14–16

Docs & limits

- OpenAPI + themed portal, guides, Redis rate limits

Done when: k6 shows 429 at the limit with headers.

PHASE 6 · DAYS 17–21

Stretch

- API keys, CSV import/export, recurring rules

Done when: duplicate import key returns the same import.

PHASE 7 · DAYS 22–25

Story

- Authz matrix test, README, docs site, ADRs, demo video

Done when: fresh clone → running in one command.

18 README, Limitations & CV

README outline

1. Banner, pitch, CI badge, link to hosted docs
2. Quick start with curl walkthrough
3. Design decisions: money, months, idempotency, limits
4. Tests and how to run them (link the authz matrix)
5. Architecture, API style guide link
6. Limitations, license

Limitations

- Single currency per workspace; no conversion.
- No bank connections or receipt scanning.
- Not deployed as a public service; runs locally via Docker.

CV entry

Qirsh — Expense & budget REST API · Node.js, TypeScript, Fastify, PostgreSQL, Redis

- Designed an OpenAPI-first REST API with Zod schemas driving validation, types and docs.
- Implemented integer minor-unit money handling and time-zone-correct monthly reports.
- Added Redis-backed rate limiting, idempotency keys, ETag concurrency control and RFC 9457 errors.
- Covered every route × role in an authorization test matrix running on real Postgres via Testcontainers.

19 Interview Preparation

Why not store money as DECIMAL?

> NUMERIC is exact in Postgres and is a valid choice. Integers were chosen so JavaScript never sees a float: `BIGINT` maps to `bigint` /safe integers, and the exponent rule lives in one module. Be ready to defend either.

Why cursor pagination instead of page numbers?

> Offset pages shift when new rows are inserted, causing duplicates or gaps, and get slower with large offsets. A cursor on `(occurred_on, id)` uses the index and is stable.

What happens if Redis goes down?

> Readiness turns red so new traffic can move elsewhere; the limiter fails open for normal routes (availability) and closed for auth routes (brute-force protection). It's a documented trade-off.

How do you avoid breaking clients?

> Additive changes only within `/v1`, the OpenAPI spec is snapshot-tested so diffs are reviewed, deprecations go in the changelog, breaking changes would mean `/v2`.

How is tenant isolation guaranteed?

> Workspace id comes from the route and is checked against membership before any handler runs, every query filters by it, and the authz matrix test proves strangers get 404 on every route.